```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddRazorPages();
builder.Services.AddSingleton<IPasswordHasher<User>, PasswordHasher<User>>();❷
var app = builder.Build();
```

❶ You will need these using directives.

❷ Register the IPasswordHasher as a service.

Next you will make some changes to Index.cshtml.cs, as shown in listing 13.9. You remove the `ILogger` from the constructor and replace it with `IPasswordHasher` along with a backing field. You also add some properties to the `PageModel`, most of which having the `BindProperty` attribute and some data annotations to control validation and display.

Listing 13.9 The modified `IndexModel` class

```
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using PasswordHasherDemo.Models;
using System.ComponentModel.DataAnnotations;

namespace PasswordHasherDemo.Pages;

public class IndexModel : PageModel
{
    private readonly IPasswordHasher<User> _passwordHasher;     ❶
    Public IndexModel(IpasswordHasher<User> passwordHasher)     ❶
    {                                                           ❶
        _passwordHasher = passwordHasher;                       ❶
    }                                                           ❶

    [BindProperty,Required,DataType(DataType.Password)]         ❷
    public string Password { get; set; }                        ❷
    [BindProperty,Required,Display(Name ="User Name")]          ❷
    public string UserName { get; set; }                        ❷
    [BindProperty,HiddenInput]                                  ❸
    public string HashedPassword { get; set; }                  ❸
    public string Result { get; set; }                          ❹
}
```

❶ Inject the IPasswordHasher instead of a logger.

❷ Add some properties for a login form.

❸ Add a property for storing the hashed password in a hidden input.

❹ Also add a property to capture the result of the validation operation.